2026년 4월 2일 목요일

Urp 4/7 미팅때 할 얘기들 …

NVIDIA 의 GTO(Greedy-then-Oldest 적용방식)

GTO 방식은 일반적으로 Oldest 워프를 찾아야 해서 comparator tree 필요 (O(N) -> 엄청난 하드웨어 낭비(정렬 로직)이 발생함.

그런데 NVIDIA는 어떻게 이 문제를 해결했지?

1. FIFO Queue 활용 : 애초에 나이순으로 워프를 줄세움
stall 발생해서 oldest 워프를 찾아야할때 comparator를 쓰지 않고 FIFO Queue에서 그냥 바로 뽑아옴 O(N) -> O(1) 으로 최적화 가능 (그냥 pop() 하면 되기 때문.) -> FIFO 를 verilog로 구현할때 문제점이 많다… (조금 더 공부해 보아야함.)

2. 고정 Warp id 부여하고 priority encoder 사용: bit vector로 ready 상태 warp 판별
16개의 warp가 있다고 치면 16비트 짜리 ready-mask를 만듦.
예시: `0100 1000 0000 0010` (1번, 11번, 14번 워프 대기 중)
위 16비트를 priority encoder에 넣으면 가장 오른쪽(왼쪽도 ok)에 있는 워프를 바로 찾아줌. (1 찾으면 바로 don’t care) .

3. Divide and conquer
core 를 여러개의 sub core 로 쪼개고 sub core 마다 warp scheduler 를 둔다. -> Configuration 조작을 통한 서브 코어 모사가능

NVIDIA는 2번과 3번을 혼합하여 사용중이라고 함. GTO + priority encoder 사용

우리는 이런 엔비디아의 gto 방식을 사용하되 뭔가 추가할 만한 것이 있을까?

1. **Two-Level (투 레벨) 구조:** 매 사이클마다 작동하는 아주 빠른 스케줄러(NVIDIA의 Priority Encoder)는 그대로 둡니다. 대신 100 사이클이나 1000 사이클에 한 번씩만 작동하는 '느리고 똑똑한 스케줄러(예: 캐시 상태 분석기)'를 상위에 둡니다. 이 상위 스케줄러가 캐시 상태를 보고 "앞으로 100 사이클 동안은 이 8개의 워프만 Priority Encoder의 Ready-mask에 들어가게 해!"라고 필터링만 해주는 방식입니다.

-> queue 를 두개로 쪼개야 되어서 현재 구조에서 수정하기에 빡셈

2. **1-bit 플래그 근사:** 캐시 용량을 정확히 수치로 추적하지 않고, 캐시 미스가 한 번이라도 크게 났던 워프는 하드웨어에 1-bit짜리 `Cache_Miss_Flag`만 1로 켜둡니다. Priority Encoder 앞단에 간단한 AND 게이트를 둬서 이 플래그가 1인 워프는 Ready-mask에서 일시적으로 꺼버리는 식입니다.

->Priority Encoder로 신호가 들어가기 직전에, 캐시 미스를 유발한 워프의 자리를 0으로 만들어버리는 **간단한 비트 연산(AND 게이트) 하나만 덧붙이면 끝납니다.** 기존 하드웨어를 전혀 해치지 않는 가장 세련된 방식입니다.
->1-bit 플래그를 언제 1로 켤 것인가?가 우리팀의 아이디어. 단순히 L1 캐시 미스 횟수를 셀 수도 있고, 메모리 대기열(MSHR)이 꽉 찼을 때 플래그를 켤 수도 있다, 즉, 하드웨어 비용(면적)은 1-bit 플래그와 AND 게이트 하나로 고정해 두고, 플래그를 제어하는 독창적인 소프트웨어적/구조적 조건만 생각해보면 될듯

논문 *: Cache-Conscious Wavefront Scheduling*
Improving GPU Performance via Large Warps and Two-Level Warp Scheduling

---

OS와 Gpu 스케줄링 철학. . ..

-os는 반응속도를 위해 i/o bound를 먼저 queue에 올림-> MLFQ(multi-level feedback queue)

Ex) 사용자 응답성을 위해 **"연산(CPU)만 주구장창 하는 놈"**의 우선순위를 낮춘다.

-gpu는 연산준비가 된 애를 우대함 "GPU 스케줄링의 핵심은 '공정성'이 아니라 'ALU(연산기)가 단 1초도 쉬지 않게 만드는 것'임.

Ex) two-level warp scheduling: 연산 처리량(Throughput)을 극대화하기 위해 **"메모리(I/O)를 기다리며 스톨(Stall)에 걸린 놈"**을 Active Pool에서 쫓아내고(Pending 상태로 강등), 연산을 당장 할 수 있는 놈을 올린다. -> 조금 더 알아봐야 할듯..

Bfs 같은 graph 계산 workload 에서는 memory 관점으로 보는 것이 유리함

-gpu는 context switch 비용이 cpu보다 훨씬 큼 -> 특정 지점에서 다른 warp로 switch하는게 일반적임. Ex) compute preemption

---

`issue` 스테이지에 적용할 수 있는 최적화 기법들

A. Opcode-Aware 스케줄링 (명령어 타입 기반 분배)

단순히 RR 로 순서를 넘기다 보면, 연속으로 ALU(정수 연산) 명령어만 선택되어서 ALU는 미어터지는데 LSU(메모리 연산)나 FPU(부동소수점 연산)는 텅텅 비어 놀고 있는 상황이 발생합니다.

- **해결책:** `ibuffer`를 훑을 때, 현재 비어있는 실행 유닛의 타입(Opcode)에 맞는 명령어를 우선적으로 쏙쏙 골라서 Issue 하는 로직을 넣습니다. (이른바 핑퐁 스케줄링)

B. 오래된 명령어 우선 처리 (Oldest-First in ibuffer)

`ibuffer`의 공간은 한정되어 있습니다. 특정 워프의 명령어가 `ibuffer`를 계속 차지하고 있으면, 앞단인 `schedule` 단계가 새로운 명령어를 가져오고 싶어도 공간이 없어서 파이프라인 전체가 멈춰버립니다.

- **해결책:** RR 대신, `ibuffer`에 가장 먼저 들어와서 오래 대기한(Oldest) 명령어에게 높은 우선순위를 주어 빨리빨리 비워냅니다. 앞서 말씀드린 **'우선순위 인코더(Priority Encoder)'** 로직을 여기에 적용하면 Verilog 구현 시 타이밍 오버헤드 없이 깔끔하게 처리할 수 있습니다.

C. Write-back 포트 충돌 예측

ALU 연산은 1 사이클 만에 끝나고, 복잡한 곱셈은 3 사이클이 걸린다고 가정해 봅시다. 만약 `issue` 단계에서 아무 생각 없이 스케줄링하면, 두 명령어의 결과가 똑같은 사이클에 레지스터 파일(Register File)로 동시에 쓰기(Write-back)를 시도하다가 충돌이 날 수 있습니다.

- **해결책:** `issue` 할 때 각 명령어의 종료 시점(Latency)을 계산해서, 같은 사이클에 Write-back 포트가 겹치지 않도록 미리 교통정리를 해줍니다.

비트 마스크 위주의 간결한 연산을 쓰시는 것이 FPGA 구현에 유리합니다

연구방향

- **`schedule()` (스케줄링 단계):** "지금 대기 중인 여러 개의 워프 중에서, **이번 사이클에 누구를 실행할지 '선택'**하는 과정"입니다. 여기서 GTO나 라운드 로빈(RR) 같은 '정책(Policy)'이 개입합니다.

- **`issue()` (이슈 단계):** "스케줄러가 **이미 선택한 워프**의 명령어를 실제 연산 유닛(ALU 등)으로 **'발사(Dispatch)'**하는 과정"입니다. 여기서는 선택된 워프의 데이터(레지스터)가 준비되었는지, 연산 유닛이 비어있는지(Hazard)만 검사합니다.

현재 Vortex의 파이프라인은 `Decode`를 거친 명령어들이 `IBUF` (Issue Buffer)라는 대기열에 쌓여있다가 `issue()` 단계를 거쳐 실제 연산기(ALU, FPU, LSU 등)로 발사되는 구조입니다.

- **현재의 문제 (단순 RR):** 만약 대기 중인 1번 워프가 정수 연산기(ALU)를 써야 하는데 ALU가 현재 꽉 차 있다면, 1번 워프는 출발하지 못하고 파이프라인이 멈춥니다(Stall). 그런데 그 뒤에 있는 2번 워프는 마침 텅텅 비어있는 부동소수점 연산기(FPU)를 써야 하는 상황이라고 해봅시다. 단순 RR 방식은 순서를 융통성 있게 바꾸지 못해서 2번 워프까지 억울하게 기다려야 합니다.

- **제안하신 최적화 (비어있는 연산기 우선 투입):** Arbiter가 현재 비어있는 연산기(예: FPU, LSU)가 무엇인지 먼저 파악하고, 대기열(IBUF)에서 그 연산기를 필요로 하는 워프를 쏙 골라서 먼저 출발시킵니다.

- **효과:** 연산기들의 **활용도(Utilization)**가 극대화되고, 파이프라인에 빈 공간(Bubble)이 생기는 것을 막아주어 전체적인 성능(IPC)이 상승합니다.

2. `schedule()` 최적화와 `issue()` 최적화의 차이점

이 두 가지는 **"해결하고자 하는 병목의 종류"**가 완전히 다릅니다. 이 차이를 논문에 잘 적으시면 아주 멋진 스토리가 됩니다.

- **`schedule()` + 1-bit 플래그 (메모리 최적화):** 워프가 캐시 미스를 내서 수백 사이클 동안 데이터를 기다려야 하는 **메모리 바운드(Memory Bound)** 상황을 해결합니다.
- **`issue()` + 비어있는 연산기 우선 (연산 최적화):** 데이터는 다 준비되었는데, 계산할 자리가 없어서 멈추는 **컴퓨트 바운드(Compute Bound) 및 구조적 해저드(Structural Hazard)** 상황을 해결합니다.

💡 **왜 `schedule()`인가?** 만약 캐시 미스를 많이 낸 워프를 `issue()` 단계에서 막아버리면 어떻게 될까요? 스케줄러는 귀중한 1 사이클을 써서 그 워프를 기껏 골라놨는데, 다음 단계에서 "어, 얘 플래그 켜져 있네? 취소!" 하고 버려버리게 됩니다. 즉, **파이프라인 1 사이클이 통째로 낭비(Bubble)** 되는 것입니다.

따라서 1-bit 플래그 로직은 `schedule()` 단계의 앞단에 들어가야 합니다. 대기 중인 워프들의 목록(Ready-mask)을 스케줄러가 확인하기 직전에, **플래그가 켜진 워프들을 미리 스케줄링 후보 명단에서 지워버려야(Masking)** 스케줄러가 안전하고 빠르게 다른 정상적인 워프를 선택할 수 있습니다.

최종

Phase 1 (메모리 타겟): 먼저 계획했던 대로 `schedule()` 단계에 1-bit 플래그(캐시 스로틀링)를 적용하여 실험을 완료합니다. (구현이 상대적으로 직관적이고 효과가 큼)

Phase 2 (연산 자원 타겟): 시간적 여유가 된다면, `issue()` 단계의 RR Arbiter를 뜯어서 **"연산기 Busy/Ready 신호를 확인하여 융통성 있게 Issue 하는 로직"**을 추가해 봅니다. (alu , fpu 등등 비어있는 곳에 넣는 느낌?) -> Trade-off:Area(면적)와 Power(전력) but 전체 성능 향상? . 즉, 여기서는 resource awarness arbiter를 구현한다는 것임.

최종 논문 스토리: "GPU의 병목을 메모리 관점(`schedule`)과 연산 자원 관점(`issue`) 양쪽에서 분석하고, 각각의 단계에 맞는 최적화 기법을 제안했다."

gpu가 cache를 쓰는 방식을 공부하자…